CareerLens — Project Architecture and API Specification

Project: CareerLens — Placement Intelligence
Purpose: Define the AWS architecture, database design, API contracts, authorization rules, file-storage flows, and deployment model.
Target audience: Student developer, reviewer, and technical evaluator
Status: Proposed implementation specification
Architecture constraint: AWS-managed/serverless services only; EC2 is not permitted.

---

1. Architecture Overview

1.1 System architecture

CareerLens is a serverless web application that helps students manage job applications, evaluate skill gaps, track interviews, and store resumes.

The Next.js frontend communicates with backend endpoints through Amazon API Gateway. AWS Lambda handles business logic, Amazon DynamoDB stores application data, Amazon Cognito manages authentication, and Amazon S3 stores private resume files.

- **flowchart TD**
- **U[Student / Job Seeker]**
- **FE[Next.js Frontend]**
- **AUTH[Amazon Cognito User Pool]**
- **API[Amazon API Gateway HTTP API]**
- **L1[Profile Lambda]**
- **L2[Applications Lambda]**

- L3[Skill Analysis Lambda]
- L4[Interviews Lambda]
- L5[Resumes Lambda]
- L6[Dashboard Lambda]
- DB[(Amazon DynamoDB)]
- S3[(Private Amazon S3 Bucket)]
- CW[Amazon CloudWatch]
-
- U --> FE
- FE <--> AUTH
- FE -->|HTTPS + JWT| API
- API --> L1
- API --> L2
- API --> L3
- API --> L4
- API --> L5
- API --> L6
- L1 --> DB
- L2 --> DB
- L3 --> DB
- L4 --> DB
- L5 --> DB
- L6 --> DB
- L5 -->|Presigned URLs| S3
- FE -->|Direct file upload/download| S3
- L1 -. Logs and metrics .-> CW
- L2 -. Logs and metrics .-> CW
- L3 -. Logs and metrics .-> CW
- L4 -. Logs and metrics .-> CW
- L5 -. Logs and metrics .-> CW

L6 -. Logs and metrics .-> CW

1.2 AWS service responsibilities

Component	Responsibility
AWS Amplify Hosting	Hosts and deploys the Next.js frontend
Amazon Cognito User Pools	Registration, login, and JWT issuance
Amazon API Gateway HTTP API	HTTPS endpoints, routing, JWT validation, throttling, and CORS
AWS Lambda	Backend business logic and validation
Amazon DynamoDB	Profiles, applications, interviews, resume metadata
Amazon S3	Private storage for PDF resumes
Amazon CloudWatch	Application logs, errors, and metrics
AWS IAM	Least-privilege permissions for Lambda and deployment

AWS SAM or AWS CDK	**Infrastructure definition and repeatable deployment**

No-EC2 rule: The application does not require EC2, a manually managed server, or a continuously running backend process. Amplify, API Gateway, Lambda, DynamoDB, Cognito, and S3 are managed services.

---

2. Request and Authentication Flow

2.1 Normal API request

1. **The user signs in through Amazon Cognito.**
2. **The frontend receives authentication tokens.**
3. **The frontend sends an HTTPS request to API Gateway with an access token.**
4. **API Gateway validates the JWT using a Cognito User Pool JWT authorizer.**
5. **API Gateway forwards authorized requests to the mapped Lambda function.**
6. **Lambda validates request data and derives the user's identity from trusted authorizer claims.**
7. **Lambda reads or writes only records belonging to that user.**
8. **API Gateway returns the response to the frontend.**

Example request header:

- **Authorization: Bearer <COGNITO_ACCESS_TOKEN>**

Content-Type: application/json

2.2 JWT authorization rules

- Configure the API Gateway JWT authorizer with the Cognito issuer URL and the appropriate audience/client identifier for the chosen token format.
- Use a valid Cognito access token for API authorization.
- Obtain the user identifier from the verified authorizer claims, normally the `sub` claim.
- Never accept `userId` from the request body as proof of identity.
- Do not treat an unverified JWT decoded by frontend JavaScript as authorization.
- Reject missing, expired, invalid, or incorrectly issued tokens.
- Validate record ownership on every read, update, delete, and download operation.

The frontend may display a user's own data, but backend authorization is the security boundary.

---

3. Lambda Function and Route Mapping

For the MVP, use separate Lambda handlers by feature. This keeps responsibilities clear while avoiding one large function containing the entire application.

Lambda function	Main responsibilities
`profile-handler`	Read and update user profile and skill inventory

`applications-handler`	Create, list, read, update, and delete applications
`skill-analysis-handler`	Compare saved job requirements with the user's skills
`interviews-handler`	Create, list, update, and delete interview records
`resumes-handler`	Issue upload/download URLs, verify uploads, and manage metadata
`dashboard-handler`	Return application counts and placement progress

All protected routes use the same Cognito JWT authorizer. A route is associated with one handler, and that handler calls shared validation, repository, and service modules.

Suggested backend structure:

- backend/
- ├── src/
- │ ├── handlers/
- │ │ ├── profile-handler.ts
- │ │ ├── applications-handler.ts
- │ │ ├── skill-analysis-handler.ts
- │ │ ├── interviews-handler.ts

- │ │ ├── resumes-handler.ts
- │ │ └── dashboard-handler.ts
- │ ├── services/
- │ ├── repositories/
- │ ├── schemas/
- │ ├── middleware/
- │ ├── utils/
- │ └── types/
- ├── tests/
- └── package.json
-
- infrastructure/
- ├── template.yaml

└── parameters/

Use AWS SDK for JavaScript v3, TypeScript, and shared modules for consistent validation and error handling.

---

4. DynamoDB Data Model

4.1 Design decision

Use one DynamoDB table named `CareerLensData` for user profiles, applications, interviews, and resume metadata.

A single-table design reduces infrastructure overhead and supports the required queries through partition/sort keys and global secondary indexes (GSIs). All records are partitioned by authenticated user, simplifying user-level isolation.

Table primary key

- Partition key: `PK` — String
- Sort key: `SK` — String

Primary key conventions

Entity	PK	SK
Profile	USER#<sub>	PROFILE
Application	USER#<sub>	APP#<applicationId>
Interview	USER#<su	INTERVIEW#<interviewId>

	b >	
Resume metadata	USER#<sub>	RESUME#<resumeId>

<sub> is the authenticated user's Cognito subject. IDs are generated by the backend, preferably as UUIDs.

4.2 Global secondary indexes

Create the following GSIs to support common access patterns without scanning the table.

I	Partition key	Sort key	Access pattern
GSI1	GSI1PK	GSI1	Applications filtered by status

		SK	
GSI2PK		GSI2SK	All applications ordered by creation time
GSI3PK		GSI3SK	Interviews ordered by interview date

Populate the index attributes only on records that need that index.

Application index attributes

- **GSI1PK = USER#<sub>#STATUS#<status>**
- **GSI1SK = <createdAt>#APP#<applicationId>**
-
- **GSI2PK = USER#<sub>#APPLICATIONS**

GSI2SK = <createdAt>#APP#<applicationId>

Interview index attributes

- **GSI3PK = USER#<sub>#INTERVIEWS**

GSI3SK = <interviewDate>#INTERVIEW#<interviewId>

Use UTC ISO 8601 timestamps, such as `2026-10-27T10:30:00.000Z`. Use a consistent sortable timestamp format for index sort keys.

4.3 Example application record

- **{**
- **"PK": "USER#cognito-sub-123",**
- **"SK": "APP#app-uuid-001",**
- **"entityType": "APPLICATION",**
- **"applicationId": "app-uuid-001",**
- **"companyName": "Example Technologies",**
- **"jobTitle": "Junior Data Engineer",**
- **"jobDescription": "Work with Python, SQL, ETL pipelines and data quality.",**
- **"requiredSkills": ["Python", "SQL", "ETL", "Data Quality"],**
- **"status": "APPLIED",**
- **"applicationDate": "2026-10-25",**
- **"notes": "Submitted through the company careers page.",**
- **"createdAt": "2026-10-25T09:00:00.000Z",**
- **"updatedAt": "2026-10-25T09:00:00.000Z",**
- **"GSI1PK": "USER#cognito-sub-123#STATUS#APPLIED",**
- **"GSI1SK": "2026-10-25T09:00:00.000Z#APP#app-uuid-001",**
- **"GSI2PK": "USER#cognito-sub-123#APPLICATIONS",**
- **"GSI2SK": "2026-10-25T09:00:00.000Z#APP#app-uuid-001"**

}

4.4 Example interview record

- **{**
- **"PK": "USER#cognito-sub-123",**
- **"SK": "INTERVIEW#interview-uuid-001",**

- "entityType": "INTERVIEW",
- "interviewId": "interview-uuid-001",
- "applicationId": "app-uuid-001",
- "roundName": "Technical Interview",
- "interviewDate": "2026-11-02T10:00:00.000Z",
- "mode": "ONLINE",
- "notes": "Revise SQL joins and Python fundamentals.",
- "createdAt": "2026-10-27T10:30:00.000Z",
- "GSI3PK": "USER#cognito-sub-123#INTERVIEWS",
- "GSI3SK": "2026-11-02T10:00:00.000Z#INTERVIEW#interview-uuid-001"

}

4.5 Example resume metadata record

- {
- "PK": "USER#cognito-sub-123",
- "SK": "RESUME#resume-uuid-001",
- "entityType": "RESUME",
- "resumeId": "resume-uuid-001",
- "fileName": "Akash_Resume.pdf",
- "objectKey": "private/cognito-sub-123/resume-uuid-001.pdf",
- "contentType": "application/pdf",
- "sizeBytes": 245760,
- "createdAt": "2026-10-27T10:30:00.000Z"

}

Store the S3 object key, not the file itself, in DynamoDB. Do not store long-lived public URLs or authentication tokens in database records.

4.6 Query rules

- Fetch a profile with `GetItem` using `PK = USER#<sub>` and `SK = PROFILE`.
- List all applications with `Query` against `GSI2`.
- List applications of a given status with `Query` against `GSI1`.
- Retrieve an individual application using its user partition key and application sort key.
- List interviews with `Query` against `GSI3`.
- List resumes by querying the user's partition and filtering the sort-key prefix `RESUME#`, or use an appropriate dedicated index if resume volume later warrants it.
- Use `Query` and `GetItem` for routine operations. Avoid `Scan` in application request paths.
- Treat GSIs as eventually consistent. For an operation requiring immediate confirmation of a write, read the base table rather than relying on a GSI.

4.7 Status values

Use a controlled enum for application status:

- SAVED
- APPLIED
- ONLINE_ASSESSMENT
- INTERVIEW
- OFFER
- REJECTED

WITHDRAWN

When status changes, update the record's `GSI1PK` and `GSI1SK` attributes as part of the same DynamoDB update.

The MVP can store the current status and its update timestamp. A full status-history timeline can be added later as separate history records if required.

---

5. Complete API Endpoint Inventory

Base URL

`https://<api-id>.execute-api.<region>.amazonaws.com/v1`

All routes below require a valid Cognito access token unless explicitly marked public.

5.1 Profile APIs

Method	Endpoint	Purpose
GET	/profile	Retrieve the current user's profile
PATCH	/profi	Update profile details and skill inventory

```
l
e
```

5.2 Application APIs

Method	Endpoint	Purpose
GET	/applications	List applications, with optional status filter
POST	/applications	Create an application
GET	/applications/{applicationId}	Retrieve one application
PATCH	/applications/{applicationId}	Update an application

DELETE	/applications/{applicationId}	Delete an application
POST	/applications/{applicationId}/skill-analysis	Calculate a skill-gap report

5.3 Interview APIs

Method	Endpoint	Purpose
GET	/interviews	List the user's interviews
POST	/interviews	Schedule an interview

PATCH	/interviews/{interviewId}	Update an interview
DELETE	/interviews/{interviewId}	Delete an interview

5.4 Resume APIs

Method	Endpoint	Purpose
GET	/resumes	List resume metadata
POST	/resumes	Create an upload intent and return a presigned upload URL

POST	/resumes/{resumeId}/complete	Verify the upload and finalize metadata
GET	/resumes/{resumeId}/download-url	Return a short-lived download URL
DELETE	/resumes/{resumeId}	Delete the resume object and metadata

5.5 Dashboard and operational APIs

Method	Endpoint	Purpose
GET	/dashboard/summary	Return application counts and placement metrics

`GET`	`/health`	Minimal operational health response; no personal data

The health route may be public because it returns no user-specific information. It should not expose internal configuration, stack traces, credentials, or database contents.

---

6. API Contracts and JSON Examples

The examples below define the expected request and response shapes. Implement the same contract in frontend TypeScript types and backend validation schemas.

6.1 Profile

`GET /profile`

Example response:

```
{
  "profile": {
    "displayName": "Akash",
    "email": "student@example.com",
    "skills": ["Python", "SQL", "Pandas"],
    "createdAt": "2026-10-20T09:00:00.000Z",
    "updatedAt": "2026-10-27T10:00:00.000Z"
  }
}
```

The email should come from a verified identity source or a trusted Cognito claim. It should not be accepted as proof of identity from an arbitrary client field.

`PATCH /profile`

Request:

- {
- "displayName": "Akash",
- "skills": ["Python", "Advanced SQL", "Pandas"]

}

Response:

- {
- "profile": {
- "displayName": "Akash",
- "skills": ["Python", "Advanced SQL", "Pandas"],
- "updatedAt": "2026-10-27T10:30:00.000Z"
- }

}

Validate allowed fields and set `updatedAt` on the backend.

6.2 Applications

`POST /applications`

Request:

- {
- "companyName": "Example Technologies",
- "jobTitle": "Junior Data Engineer",
- "jobDescription": "Python, SQL, ETL and data quality.",
- "requiredSkills": ["Python", "SQL", "ETL", "Data Quality"],
- "status": "APPLIED",
- "applicationDate": "2026-10-27",
- "notes": "Applied through the careers portal."

}

Successful response — `201 Created`:

```
{
  "application": {
    "applicationId": "app-uuid-001",
    "companyName": "Example Technologies",
    "jobTitle": "Junior Data Engineer",
    "status": "APPLIED",
    "applicationDate": "2026-10-27",
    "createdAt": "2026-10-27T10:30:00.000Z",
    "updatedAt": "2026-10-27T10:30:00.000Z"
  }
}
```

The backend generates `applicationId`, timestamps, and all database key attributes. The client must not provide `PK`, `SK`, `GSI1PK`, `GSI1SK`, `GSI2PK`, or `GSI2SK`.

`GET /applications`

Supported query parameters:

- GET /applications?status=APPLIED&limit=20
- GET /applications?limit=20&cursor=<opaque-token>

GET /applications?status=INTERVIEW&limit=10

- No status filter: query `GSI2`.
- With a status filter: query `GSI1`.
- `limit` is the maximum number of returned records, capped by the backend.
- `cursor` represents an encoded DynamoDB `LastEvaluatedKey`.

Example response:

- {
- "items": [
- {
- "applicationId": "app-uuid-001",
- "companyName": "Example Technologies",
- "jobTitle": "Junior Data Engineer",
- "status": "APPLIED",
- "applicationDate": "2026-10-27",
- "createdAt": "2026-10-27T10:30:00.000Z"
- }
-],
- "nextCursor": null

}

`GET /applications/{applicationId}`

Response:

- {
- "application": {
- "applicationId": "app-uuid-001",
- "companyName": "Example Technologies",
- "jobTitle": "Junior Data Engineer",
- "jobDescription": "Python, SQL, ETL and data quality.",
- "requiredSkills": ["Python", "SQL", "ETL", "Data Quality"],
- "status": "APPLIED",
- "applicationDate": "2026-10-27",
- "notes": "Applied through the careers portal.",
- "createdAt": "2026-10-27T10:30:00.000Z",
- "updatedAt": "2026-10-27T10:30:00.000Z"
- }

}

`PATCH /applications/{applicationId}`

Request:

- **{**
- **"status": "INTERVIEW",**
- **"notes": "Technical interview scheduled."**

}

Response:

- **{**
- **"application": {**
- **"applicationId": "app-uuid-001",**
- **"status": "INTERVIEW",**
- **"notes": "Technical interview scheduled.",**
- **"updatedAt": "2026-10-28T08:00:00.000Z"**
- **}**

}

Validate the supplied fields and update the status index attributes if the status changes. Use a conditional update so a missing or non-owned record cannot be modified.

`DELETE /applications/{applicationId}`

Successful response — `204 No Content`, with an empty body.

If the application has associated interviews or analysis data, define and implement a consistent cleanup policy. For the MVP, delete or explicitly retain related records according to a documented rule; do not leave orphaned records unintentionally.

6.3 Skill-gap analysis

`POST /applications/{applicationId}/skill-analysis`

This endpoint compares the authenticated user's saved skill inventory with the application's required skills. It does not require an external AI service.

Example response:

- **{**
- **"applicationId": "app-uuid-001",**
- **"matchedSkills": ["Python", "SQL"],**
- **"missingSkills": ["ETL", "Data Quality"],**
- **"matchCount": 2,**
- **"requiredSkillCount": 4,**
- **"matchPercentage": 50,**
- **"method": "NORMALIZED_SKILL_MATCH",**
- **"disclaimer": "This is a rule-based skill comparison, not a hiring probability."**

}

Suggested deterministic matching rules:

1. **Trim whitespace and normalize case.**
2. **Remove duplicate skill entries.**
3. **Apply a small, explicit alias map where appropriate, such as `postgres` to `postgresql`.**
4. **Compare normalized required skills with normalized user skills.**
5. **Calculate `matchPercentage = matchedSkillCount / requiredSkillCount × 100`.**
6. **If no required skills exist, return a validation error or a clearly defined "not available" result rather than dividing by zero.**

Do not claim that this percentage predicts hiring success.

6.4 Interviews

`POST /interviews`

Request:

- {
- "applicationId": "app-uuid-001",
- "roundName": "Technical Interview",
- "interviewDate": "2026-11-02T10:00:00.000Z",
- "mode": "ONLINE",
- "notes": "Revise SQL joins and Python fundamentals."

}

Response — `201 Created`:

- {
- "interview": {
- "interviewId": "interview-uuid-001",
- "applicationId": "app-uuid-001",
- "roundName": "Technical Interview",
- "interviewDate": "2026-11-02T10:00:00.000Z",
- "mode": "ONLINE",
- "notes": "Revise SQL joins and Python fundamentals.",
- "createdAt": "2026-10-27T10:30:00.000Z"
- }

}

Before creating the interview, verify that `applicationId` belongs to the authenticated user.

`GET /interviews`

Example:

- {
- "items": [
- {

- "interviewId": "interview-uuid-001",
- "applicationId": "app-uuid-001",
- "roundName": "Technical Interview",
- "interviewDate": "2026-11-02T10:00:00.000Z",
- "mode": "ONLINE"
- }
-],
- "nextCursor": null

}

`PATCH /interviews/{interviewId}`

Request:

- {
- "interviewDate": "2026-11-03T10:00:00.000Z",
- "notes": "Interview rescheduled."

}

`DELETE /interviews/{interviewId}`

Return `204 No Content` when deletion succeeds.

For both update and delete, locate the record under the authenticated user's partition key.

6.5 Resumes: upload intent

`POST /resumes`

Request:

- {
- "fileName": "Akash_Resume.pdf",
- "contentType": "application/pdf",
- "sizeBytes": 245760

```
}
```

Successful response — `201 Created`:

```
{
  "resumeId": "resume-uuid-001",
  "uploadUrl": "https://example-presigned-upload-url",
  "method": "PUT",
  "requiredHeaders": {
    "Content-Type": "application/pdf"
  },
  "expiresInSeconds": 300

}
```

The URL shown is illustrative, not a working URL. The backend creates a server-generated S3 key and signs a short-lived upload URL. Create a pending upload record or upload intent so that completion can be validated.

6.6 Resumes: complete upload

`POST /resumes/{resumeId}/complete`

Request:

```
{}
```

The backend checks that the pending resume belongs to the authenticated user, verifies the corresponding S3 object exists, checks its size and metadata, and finalizes the DynamoDB record.

Response:

```
{
  "resume": {
    "resumeId": "resume-uuid-001",
    "fileName": "Akash_Resume.pdf",
```

- "contentType": "application/pdf",
- "sizeBytes": 245760,
- "status": "READY",
- "createdAt": "2026-10-27T10:30:00.000Z"
- }

}

Do not mark the resume as ready just because the frontend reports a successful upload. Verify the object from the backend.

6.7 Resumes: list and download

`GET /resumes`

Response:

- {
- "items": [
- {
- "resumeId": "resume-uuid-001",
- "fileName": "Akash_Resume.pdf",
- "contentType": "application/pdf",
- "sizeBytes": 245760,
- "status": "READY",
- "createdAt": "2026-10-27T10:30:00.000Z"
- }
-]

}

`GET /resumes/{resumeId}/download-url`

Response:

- {
- "downloadUrl": "https://example-presigned-download-url",

- "expiresInSeconds": 300

}

The URL is generated only after verifying that the resume belongs to the user. It should expire quickly and must not be stored as a permanent URL in DynamoDB.

`DELETE /resumes/{resumeId}`

Delete the S3 object and associated metadata. Handle partial failures safely: if the object is already missing, the backend should be able to reconcile metadata without exposing another user's data. Return `204 No Content` after the deletion workflow succeeds.

6.8 Dashboard

`GET /dashboard/summary`

Example response:

- {
- "totalApplications": 12,
- "byStatus": {
- "SAVED": 2,
- "APPLIED": 5,
- "ONLINE_ASSESSMENT": 1,
- "INTERVIEW": 2,
- "OFFER": 1,
- "REJECTED": 1,
- "WITHDRAWN": 0
- },
- "upcomingInterviews": 2,
- "lastUpdatedAt": "2026-10-27T10:30:00.000Z"

}

The dashboard must count only records belonging to the authenticated user. For a small MVP dataset, counts can be calculated through targeted queries. Avoid scanning the entire table on every dashboard request. If usage grows, add maintained aggregate counters or a dedicated summary item.

6.9 Health check

```
GET /health
```

Response:

- **{**
- **"status": "ok"**

}

This endpoint reports only basic service health. Do not expose credentials, environment variables, stack traces, internal table names, or private user information.

---

7. S3 Upload and Download Architecture

7.1 Upload sequence

- **sequenceDiagram**
- **participant F as Next.js Frontend**
- **participant A as API Gateway**
- **participant L as Resumes Lambda**
- **participant D as DynamoDB**
- **participant S as Private S3 Bucket**
-
- **F->>A: POST /resumes with JWT**
- **A->>L: Authorized request**
- **L->>D: Create pending upload intent**
- **L-->>F: Upload URL + resumeId**

- F->>S: PUT PDF using presigned URL
- S-->>F: Upload result
- F->>A: POST /resumes/{id}/complete
- A->>L: Authorized completion request
- L->>S: Verify object metadata
- L->>D: Mark resume READY

L-->>F: Final resume metadata

7.2 Upload rules

- Keep the S3 bucket private and enable S3 Block Public Access.
- Allow PDF files only for the initial MVP.
- Apply a maximum size of 5 MB.
- Generate object keys on the backend; do not use user-provided filenames as the S3 key.
- Bind the presigned URL to the intended object key and content type.
- Keep URL expiration short, for example five minutes.
- Validate the actual stored object's size and metadata during completion.
- Do not trust the file extension alone as proof of file type.
- Configure S3 CORS for the exact frontend origin and the required `PUT`/`GET` methods and headers.
- Never place AWS access keys or long-lived AWS credentials in frontend code.
- Use a Lambda execution role with only the S3 permissions required for the resume prefix and bucket.

7.3 Download sequence

1. The frontend calls `GET /resumes/{resumeId}/download-url` with a valid JWT.
2. The Lambda handler retrieves metadata using the authenticated user's partition key.

3. If the record does not belong to that user, the operation is rejected.
4. Lambda generates a short-lived S3 presigned GET URL.
5. The frontend downloads the PDF directly from S3.

This design avoids routing large file transfers through API Gateway or Lambda.

7.4 Failure handling

- If upload fails, leave the upload intent pending and allow a retry while the intent is valid.
- If completion finds no object, return a conflict or validation error and do not mark the record `READY`.
- Expire or clean up abandoned pending uploads.
- If an S3 object is uploaded but metadata finalization fails, allow the completion endpoint to be retried safely.
- Consider an S3 lifecycle rule for temporary objects or abandoned uploads if the implementation creates a dedicated temporary prefix.

---

8. Authorization and Data Protection

8.1 Authorization matrix

Resource	Access rule
Profile	Only the authenticated owner
Applications	Only the authenticated owner

Skill analysis	Only against the owner's profile and application
Interviews	Only the authenticated owner; linked application must also be owned
Resume metadata	Only the authenticated owner
Resume upload/download URLs	Only the authenticated owner
Dashboard	Only the authenticated owner's data
Health	Public, minimal status only

8.2 Backend authorization pattern

For every user-owned operation:

1. Read the authenticated `sub` claim from the API Gateway authorizer context.
2. Construct the user partition key on the backend.
3. Query or update the record using that partition key and the required sort key.
4. Use DynamoDB conditional expressions where appropriate.
5. Return a consistent error if the resource is missing or is not accessible to the user.

For example, an application lookup must use:

- **PK = USER#<authenticated-sub>**

  SK = APP#<applicationId>

Do not query the table by an untrusted application ID alone and then assume the caller owns the result.

8.3 Additional security requirements

- **Use HTTPS for frontend-to-API and frontend-to-S3 transfers.**
- **Restrict API Gateway CORS to the deployed frontend origin.**
- **Use separate least-privilege IAM execution roles for Lambda functions.**
- **Keep S3 private and DynamoDB inaccessible directly from the browser.**
- **Validate all request bodies and path parameters.**
- **Avoid logging access tokens, presigned URLs, resume contents, or sensitive user data.**
- **Use conditional writes to reduce accidental overwrites and race conditions.**
- **Keep production secrets out of source control and frontend environment variables.**
- **Apply throttling and sensible payload-size limits.**
- **Configure CloudWatch log retention and alarms for repeated failures.**

---

9. Pagination, Filtering, and Sorting

9.1 Pagination contract

Use cursor-based pagination for application and interview lists.

Example:

- **GET /applications?limit=20**

GET /applications?limit=20&cursor=<opaque-token>

The Lambda handler should:

1. **Validate `limit` and cap it at a safe maximum, such as 50.**
2. **Decode the cursor into the DynamoDB `ExclusiveStartKey`.**
3. **Execute the relevant `Query`.**
4. **Return at most the requested number of results.**
5. **Encode the returned `LastEvaluatedKey` into `nextCursor`.**

Example response:

- **{**
- **"items": [],**
- **"nextCursor": "eyJleGFtcGxlIjoiY3Vyc29yIn0"**

}

The cursor above is illustrative. The actual cursor must be produced by the backend.

Treat cursors as opaque values, validate their structure and size, and bind them to the expected query/index and authenticated user. A cursor is not an authorization mechanism. Every request must still use the caller's authenticated identity.

9.2 Sorting

- **Applications: newest first by `GSI2SK`.**
- **Status-filtered applications: newest first by `GSI1SK`.**
- **Interviews: earliest upcoming date first by `GSI3SK`.**

- **Use reverse query order when the UI needs newest-first records for a descending date index.**

Use DynamoDB key ordering rather than loading all records and sorting them in Lambda.

---

10. Error Contract

All API errors should use the same JSON structure.

- **{**
- **"error": {**
- **"code": "VALIDATION_ERROR",**
- **"message": "The request contains invalid fields.",**
- **"details": [**
- **{**
- **"field": "companyName",**
- **"message": "Company name is required."**
- **}**
- **],**
- **"requestId": "example-request-id"**
- **}**

}

10.1 HTTP status mapping

Status	Meaning	Example
`200 OK`	Successful read or update	Profile retrieval

`201 Created`	Resource created	New application
`204 No Content`	Successful deletion	Delete application
`400 Bad Request`	Invalid input or cursor	Invalid status value
`401 Unauthorized`	Missing or invalid token	Expired JWT
`403 Forbidden`	Authenticated caller lacks permission	Disallowed operation
`404 Not Found`	Resource does not exist or is not accessible	Unknown application ID
`409 Conflict`	Resource state conflicts with the request	Upload not completed

`413 Payload Too Large`	Request exceeds the allowed size	Oversized upload intent
`429 Too Many Requests`	Rate limit exceeded	Excessive API requests
`500 Internal Server Error`	Unexpected server failure	Unhandled backend exception
`503 Service Unavailable`	Temporary service failure	Dependency unavailable

API Gateway may generate some authentication or throttling responses before a Lambda handler runs. Configure gateway-level error responses where possible so the frontend can handle them consistently.

10.2 Error-handling rules

- Never return raw stack traces or AWS SDK error objects.
- Log the internal exception with a request identifier.
- Return safe, actionable messages to the client.
- Use `400` for schema-validation failures.
- Use `404` for resources that are missing or intentionally hidden to prevent ownership enumeration.

- Use `409` for conflicts with the current resource state.
- Do not retry non-idempotent operations blindly.
- Make upload completion safe to retry.

---

11. Deployment Architecture — No EC2

11.1 Environments

Use two environments if time permits:

- Development: local Next.js frontend and a development AWS environment.
- Production/demo: deployed frontend and dedicated AWS resources used for evaluation.

For a time-constrained hiring assessment, a single development AWS environment can be deployed first, then promoted to the demo environment after testing.

11.2 Deployment flow

- flowchart TD
- G[Git Repository]
- CI[Build and Test]
- INFRA[AWS SAM or CDK Deployment]
- COG[Amazon Cognito]
- API[API Gateway]
- LAM[AWS Lambda Functions]
- DB[(DynamoDB)]
- S3[(Private S3 Bucket)]
- HOST[AWS Amplify Hosting]
- USER[Browser]
-
- G --> CI
- CI --> INFRA

- INFRA --> COG
- INFRA --> API
- INFRA --> LAM
- INFRA --> DB
- INFRA --> S3
- CI --> HOST
- HOST --> USER
- USER --> API

USER --> COG

11.3 Recommended deployment sequence

1. Prepare the repository: keep `frontend/`, `backend/`, `infrastructure/`, and `docs/` clearly separated.
2. Create infrastructure as code: define Cognito, API Gateway, Lambda, DynamoDB, S3, IAM roles, and required environment variables with AWS SAM or CDK.
3. Deploy backend resources: deploy the table, bucket, functions, API routes, JWT authorizer, and CORS configuration.
4. Configure Cognito: create the user pool and app client, configure sign-in, and test token issuance.
5. Configure frontend environment: set the API base URL, Cognito user pool ID, client ID, and AWS Region.
6. Deploy Next.js: connect the repository to Amplify Hosting and configure the production build.
7. Run smoke tests: register/login, create an application, update its status, run skill analysis, upload a PDF, and download it.
8. Review monitoring: check CloudWatch logs, Lambda errors, API Gateway responses, and S3 access behavior.

11.4 Environment configuration

Frontend environment variables:

- NEXT_PUBLIC_APP_NAME=CareerLens
- NEXT_PUBLIC_API_BASE_URL=https://your-api-id.execute-api.your-region.amazonaws.com/v1
- NEXT_PUBLIC_COGNITO_USER_POOL_ID=your-user-pool-id
- NEXT_PUBLIC_COGNITO_CLIENT_ID=your-app-client-id

NEXT_PUBLIC_AWS_REGION=your-region

Only public configuration belongs in `NEXT_PUBLIC_*` variables. Never place AWS secret access keys in the frontend.

Backend Lambda environment variables:

- DYNAMODB_TABLE_NAME=CareerLensData
- S3_RESUME_BUCKET=your-private-resume-bucket
- APP_ENV=production

LOG_LEVEL=INFO

The AWS Region should be configured through the deployment environment. IAM roles should supply service permissions instead of embedding credentials in configuration files.

11.5 Infrastructure and deployment checklist

- Cognito User Pool and app client are configured.
- API Gateway JWT authorizer validates the correct issuer and audience.
- Every protected route is attached to the authorizer.
- Lambda functions have least-privilege IAM permissions.
- DynamoDB table and GSIs match the documented key schema.
- S3 Block Public Access is enabled.
- S3 CORS allows only the necessary frontend origin and methods.
- Resume upload size and type validation are implemented.

- **API Gateway throttling and request validation are configured where supported.**
- **Frontend environment variables use the correct deployment values.**
- **CloudWatch logs are available without leaking sensitive information.**
- **Production resources are defined in infrastructure as code.**
- **No EC2 instance or long-running manually managed backend is required.**

---

12. Implementation Priorities

For a student hiring assessment, implement the features in this order:

1. **Cognito authentication and protected API calls.**
2. **Profile retrieval and skill inventory.**
3. **Application create/list/update/delete.**
4. **DynamoDB indexes and pagination.**
5. **Skill-gap analysis.**
6. **Dashboard counts.**
7. **Interview scheduling and listing.**
8. **Private S3 resume upload and download.**
9. **Error handling, authorization tests, and deployment verification.**

If time is limited, prioritize a reliable end-to-end application workflow before adding reminders, advanced analytics, or a complex status-history system.

---

13. Definition of Done

The architecture and API implementation are ready for assessment when:

- **Users can sign in and only access their own records.**
- **All documented routes have a consistent request/response contract.**
- **Applications can be created, listed, edited, and deleted.**
- **Status filters use the correct DynamoDB index.**
- **Pagination works without table scans.**
- **Skill-gap analysis returns deterministic, explainable results.**
- **Interviews are linked only to applications owned by the same user.**
- **Resume uploads use short-lived presigned URLs and private S3 storage.**
- **Resume downloads verify ownership before issuing a URL.**
- **Errors use the documented JSON format.**
- **The frontend and backend are deployed and demonstrable without EC2.**
- **Core flows and authorization failures have automated or repeatable manual tests.**

Architecture summary: CareerLens uses Next.js on Amplify Hosting, Cognito for authentication, API Gateway for protected HTTPS routes, Lambda for business logic, DynamoDB for user-owned structured data, S3 for private resume files, and CloudWatch for monitoring. This provides a compact, scalable serverless foundation suitable for the MVP and a hiring-assessment demonstration.